

Overview

The matching engine developed can handle up to 40 clients which interact with the engine through buy, sell and cancel orders. The active order interacts with orders in the order book, if the order cannot be completed, it is then added to the order book. The architecture of our engine matching is described below.

Proposed Concurrency Level: Phase-Level Concurrency

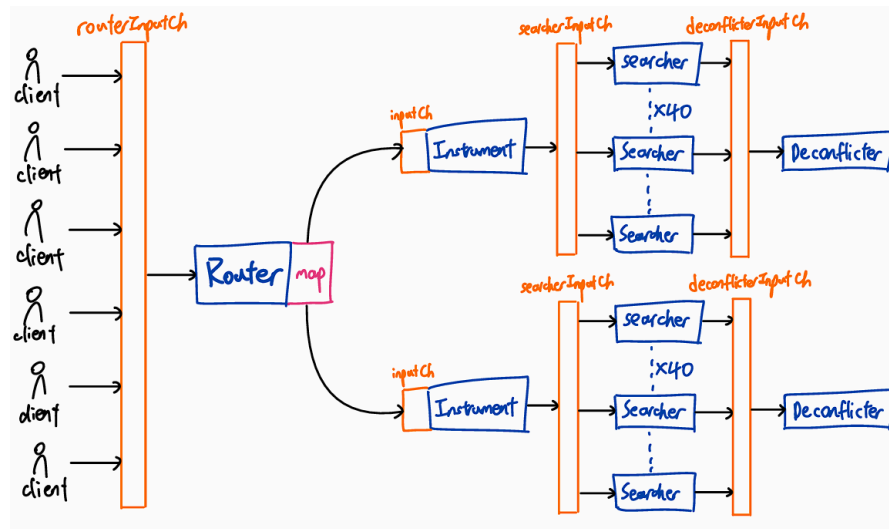


Diagram 1: Overarching system architecture

Router

The central router receives orders from multiple parallel clients through an **unbuffered** input channel in a **fan in** manner. Each client sends orders to this router, which maintains a **map** linking instruments to their corresponding goroutines. When an order arrives, the router checks the map: If a goroutine for that instrument already exists, the order is routed directly to it. If not, a new **instrument** goroutine is spawned, ensuring that every instrument is processed independently. Hence, the router **fans out** the orders it receives to their particular instrument.

This router operates using a **for-select** loop, processing orders sequentially. Although this sequential operation might briefly block incoming orders, it is necessary to synchronise this step amongst all the clients to prevent the creation of duplicate **instrument** goroutines. Given that the system will have a maximum of 40 concurrent clients, this sequential processing does not significantly impede overall concurrency.

Instrument Goroutines

Each **instrument** is managed by its own dedicated goroutine, which only processes orders for that particular instrument. This design achieves **instrument-level concurrency**. Within this goroutine, incoming orders are **fanned out** through unbuffered channels to 40 concurrent searcher goroutines, each responsible for handling their order matching process. The choice of 40 ensures optimal handling of the worst-case scenario, where all clients (up to 40) submit orders of the same type simultaneously. The design employs a **phase-level concurrency** as

multiple orders can be matched concurrently. However, only 1 type of active order (either buy or sell) can be processed at a time, while orders of the opposite type are **queued**. Once all active orders of the current type have been processed, the system switches to match the queued orders.

Order Matching Pipeline

Order matching is one of the most resource intensive steps, as it requires scanning a potentially large number of resting orders. Additionally, updating the resting order book is inherently sequential and must be **synchronised** to ensure consistency. To optimise this process, the order matcher follows a **pipeline** approach with three distinct stages:

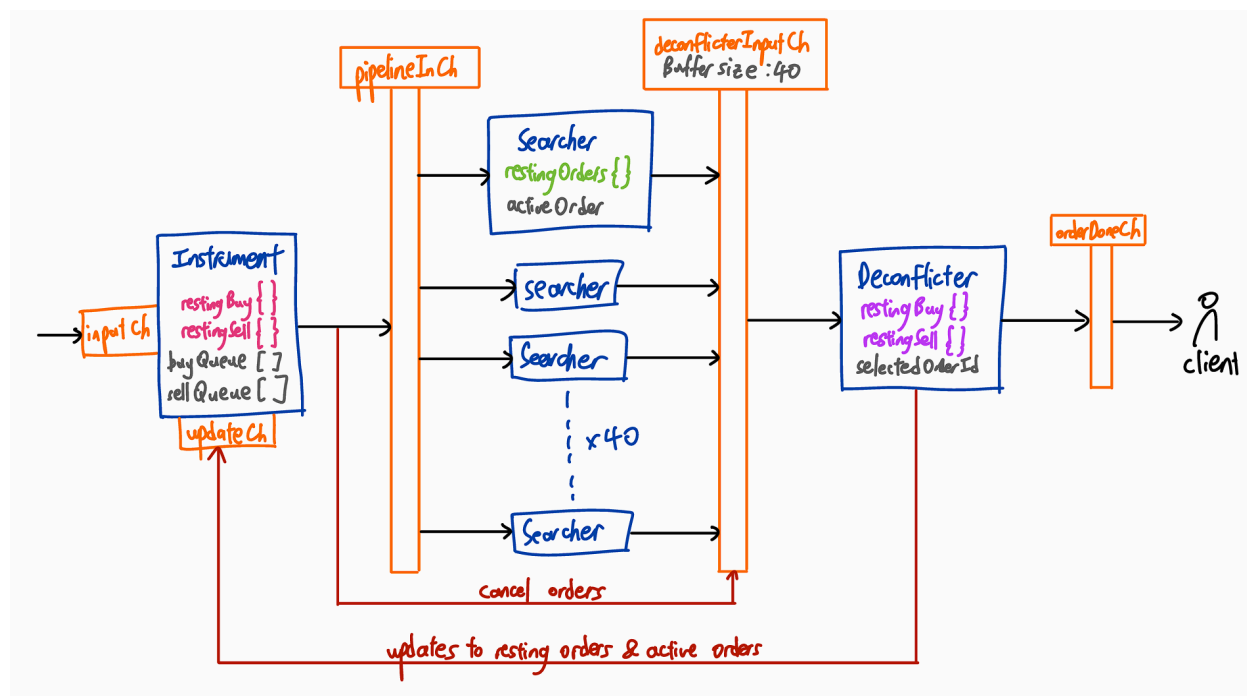


Diagram 2: Pipeline architecture

1. Searching

Once the active orders have been **fanned out**, each **searcher** goroutine begins by creating a copy of the global resting orders **map** and then works concurrently to find the best match for its active order. Since only orders on the same side (buy or sell) are processed simultaneously, the optimal resting order remains valid unless its quantity has been reduced to zero by other matches in parallel **searcher** goroutines. Any such conflicts are resolved by the deconflicting stage below.

2. Deconflicting

Once a **searcher** goroutine identifies an optimal match, it sends a matching request to the **deconflicter** goroutine via a buffered channel (buffer size of 40 to prevent blocking). All **searcher** goroutines send their matching requests through this shared channel, **fanning into** the deconflicter sequentially. As the single source of truth, the deconflicter maintains its own authoritative copy of the resting buy and sell **maps**. This

sequential processing ensures accurate count updates and resolves conflicts arising from simultaneous, illegal modifications to resting orders. If a conflict occurs, the `deconflicter` re-dispatches the active order to the instrument goroutine for further processing. Given that there will be a maximum of 40 matching requests at once, this sequential processing does not significantly impede concurrency.

3. Updating

After the `deconflicter` has processed a matching request, an updated resting order is sent back to the `instrument` goroutine via an update channel. If an active order still has a remaining count after execution, it is re-dispatched for further processing. This sequential update phase ensures that all changes to the resting orders occur in a consistent and conflict-free manner.

Cancellation & Order Book Logging

Cancellation orders bypass the full matching pipeline. At the instrument level, a *map* linking order IDs to order types enables immediate processing of cancel requests. These orders skip the search process and are directly forwarded to the `deconflicter`, which attempts to remove the resting order, provided its count is not zero. The sequential nature of the input channel to the `deconflicter` ensures that all modifications to the resting order book are legally enforced.

Client Side Completion Signal

To signal the end of the order processing pipeline, an `orderDoneCh` channel is passed through the system from the engine to the client. When this channel is closed, it signals that the order has been fully processed. The client goroutine, upon detecting the closed channel, unblocks and can safely submit the next order.

Limitations/Improvements

The matching engine can only match one type of active order (buy or sell) at a time, potentially blocking the other type for an extended period if new orders of the active type keep arriving. This can be mitigated using a turnstile mechanism, where each order type is allotted a maximum time slice to ensure fair processing.

Go patterns and Data structures used

The system architecture leverages Go's pipeline and fan-in/fan-out patterns, as described above. Maps and lists were used to track both active and resting sell orders.

Testing Methodology

The same Python script from Assignment 1 was used to generate randomised test cases, gradually increasing the number of clients while ensuring order requests adhered to the assignment rules. To scale up test case generation, ChatGPT was used to convert the Python code to Go (`scripts/gen_tests_std.go`). As in Assignment 1, `scripts/test.sh` was used to execute all test cases, with an additional check for potential memory leaks.